Network Layer - II

Congestion control

QoS

Topics

1. **Congestion Control Algorithms:**
   - introduction
   - General techniques for Congestion Control
   - Congestion Prevention Policies
   - Congestion Control in Virtual-Circuit Subnets
   - Congestion Control in Datagram Subnets
   - Load Shedding, Jitter Control

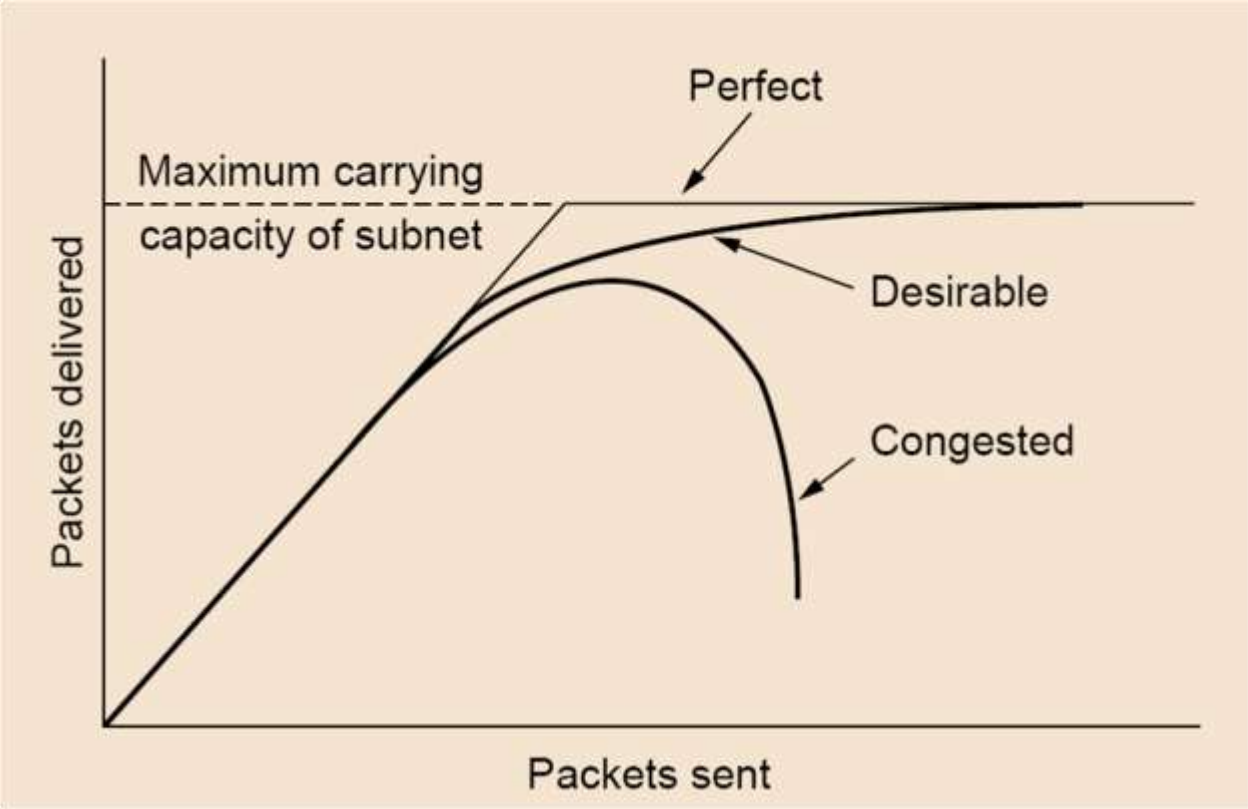
2. **Quality Of Service:**
   1. Requirements
   2. Techniques for Achieving Good Quality of Service

Congestion Control Algorithms


Traffic congestion

What is congestion?

When too many packets are present in the subnet, performance degrades. This situation is called **congestion**

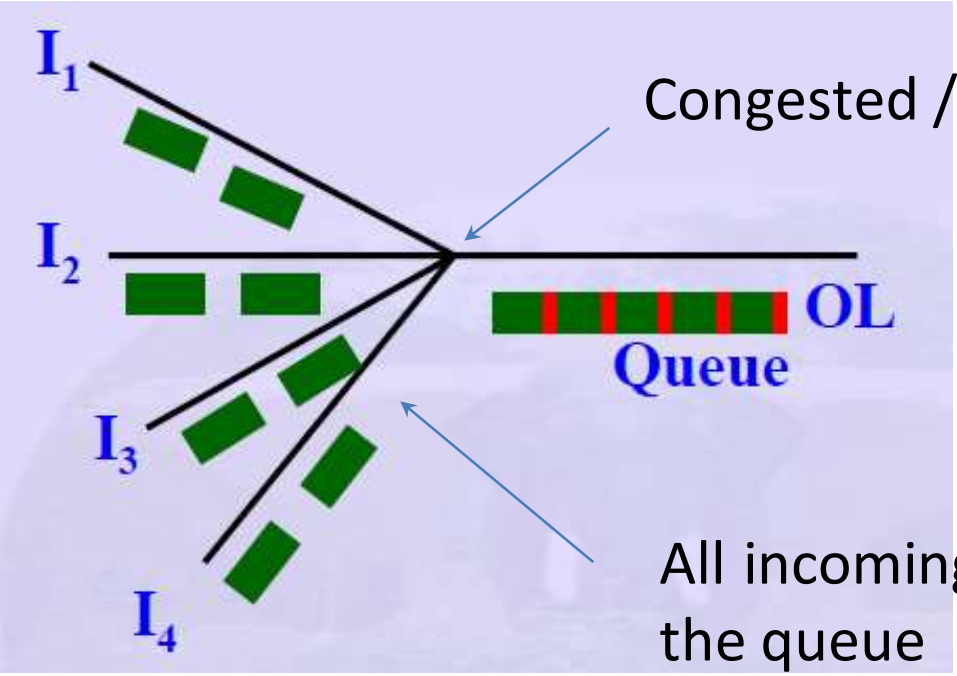


At very high trafffic, performance collapses completely and almost no packets are delivered.

Reasons for congestion

1. Insufficient memory at routers

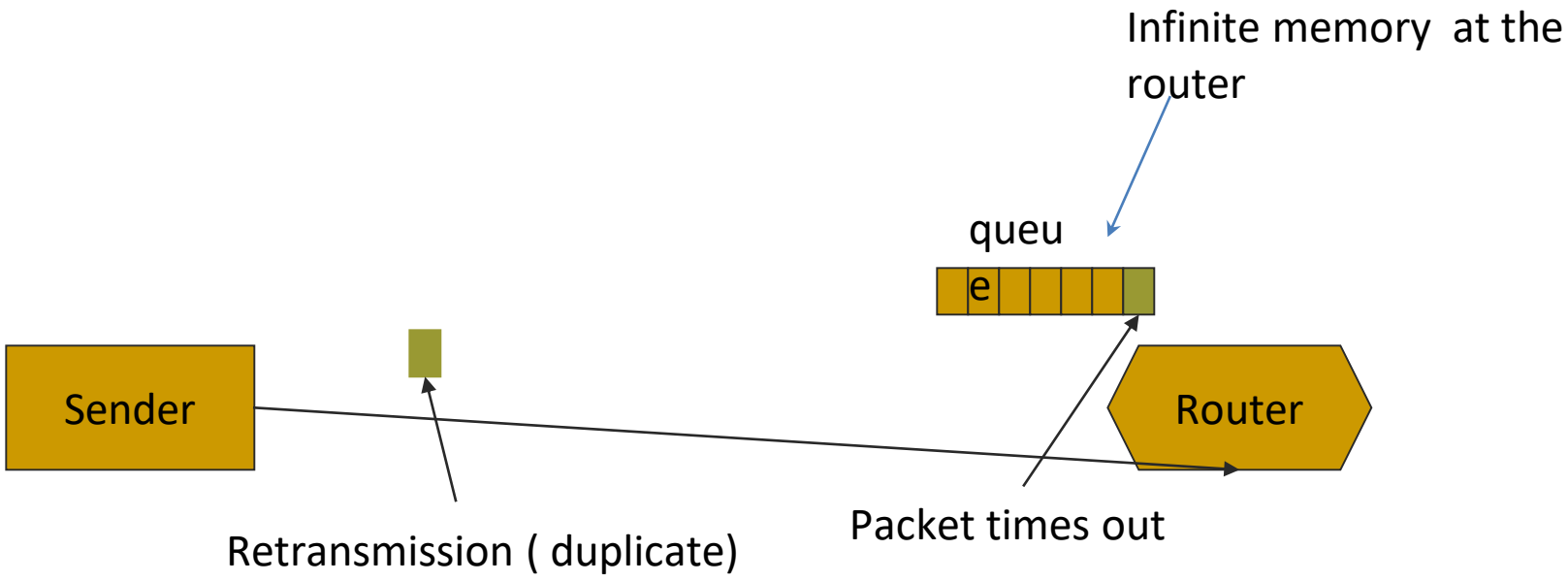
- If there is insufficient memory to hold all packet, packets will be lost.



Reasons for congestion

1. Insufficient memory at routers

- But adding more memory will make congestion worse.
- By the time a packet gets to the front of the queue , it may time out, so this will cause retransmissions. It will increase the load all the way to the destination

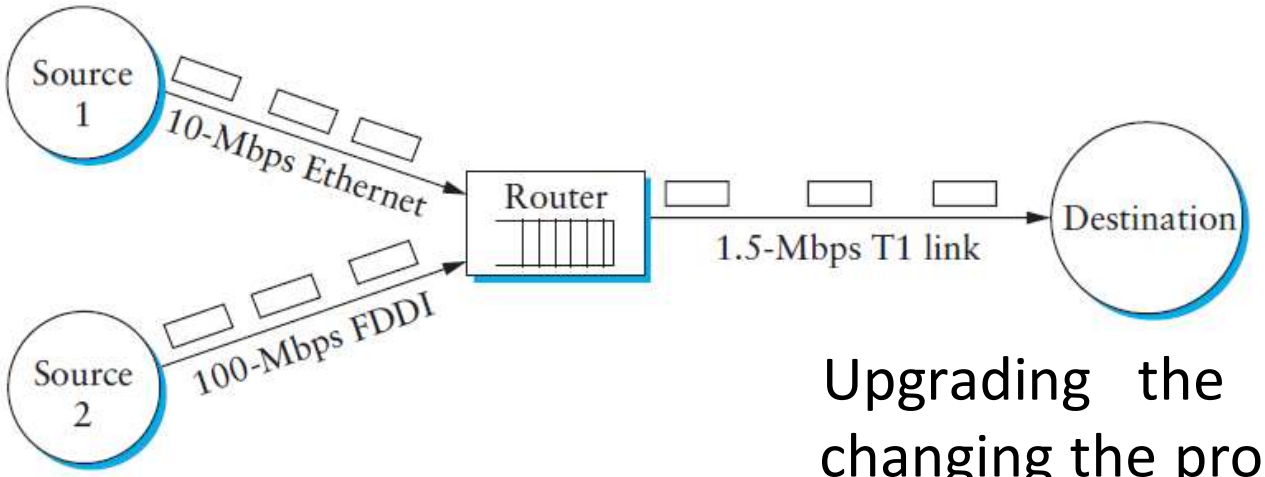


Reasons for congestion

2. If routers have slow processors

- If the routers' CPUs are slow , queues can build up, even though there is excess line capacity.

3. low-bandwidth outgoing lines



Upgrading the lines but not changing the processors, or vice versa, often helps a little

Congestion control vs. Flow control

Congestion control

- Ensures that subnet is able to carry the offered traffic.
- **Global issue -** involves the behavior of all the hosts, all the routers and other related factors

Flow control

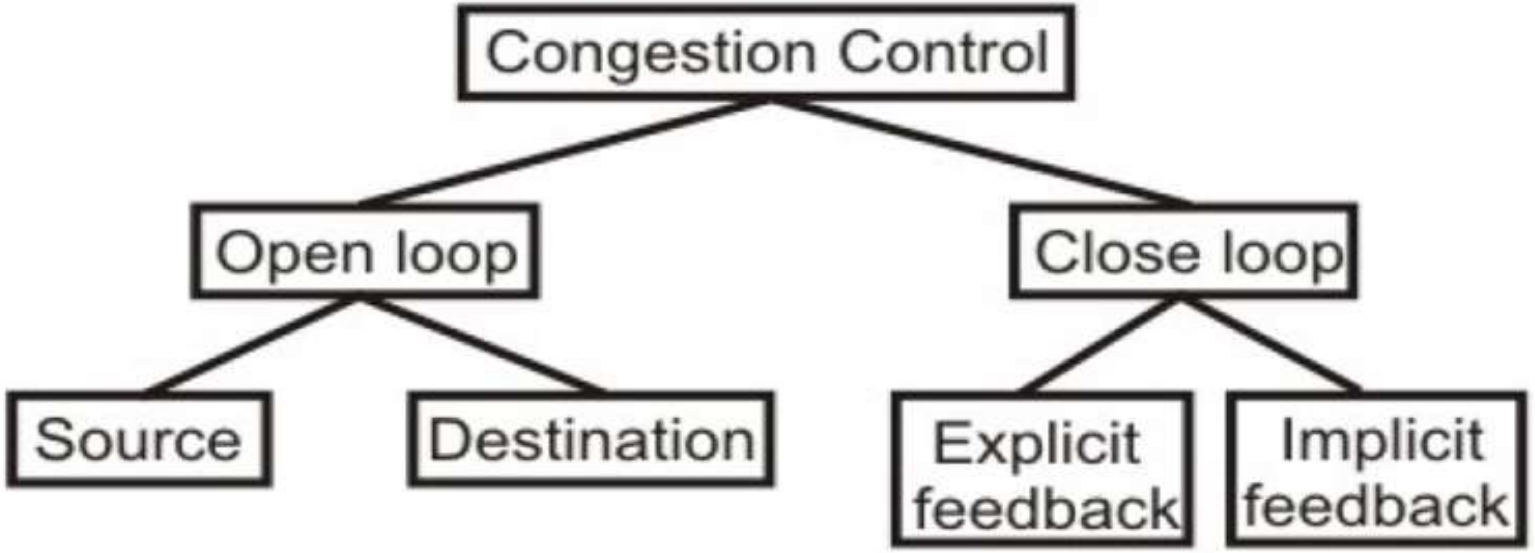
- makes sure that a fast sender cannot continually transmit data faster than the receiver is able to receive it.
- **Local Issue** - frequently involves some ***direct*** feedback from the receiver to the sender (**point to point traffic**)

Congestion Control Technique

1. Network Provisioning
2. Traffic Aware Routing
3. Admission Control
4. Traffic Throttling
5. Load Shedding

Congestion Control Techniques

Network provisioning, traffic aware routing and Admission control are covered under this



1. **Open loop :** Prevent or avoid congestion, ensuring that the system(network) never enters a Congested State (static)
2. **Close loop :**allow system to enter congested state, detect it, and remove it. (dynamic)

Open loop solutions(1)

- Attempt to solve the problem by a good design
- Once system is up and running, midcourse corrections are not made
- Further divided into ones that **act at the source** versus ones that **act at the destination**

Open loop solutions(2)

Essential points for Open Loop solutions are

- deciding when to accept new traffic
- when to discard which packets
- making scheduling (packets from queues) decisions.

- All of these decisions are based on a sensible system design
  (do not depend on the current network state.)

Closed loop solutions(1)

- Based on the concept of feedback.

- This approach can be divided into 3 steps

1.Monitor the system to detect when and where congestion occurs.
2.To pass this information to the places where actions can be taken.
3. Adjust the system operation to correct the problem.

Closed loop solutions(2)

1.monitor the system to detect any congestion

Metrics used to monitor for congestion

1. Percentage of discarded packets
2. Average queue lengths
3. Number of packets that time out and are retransmitted
4. The average packet delay

Closed loop solutions (3)

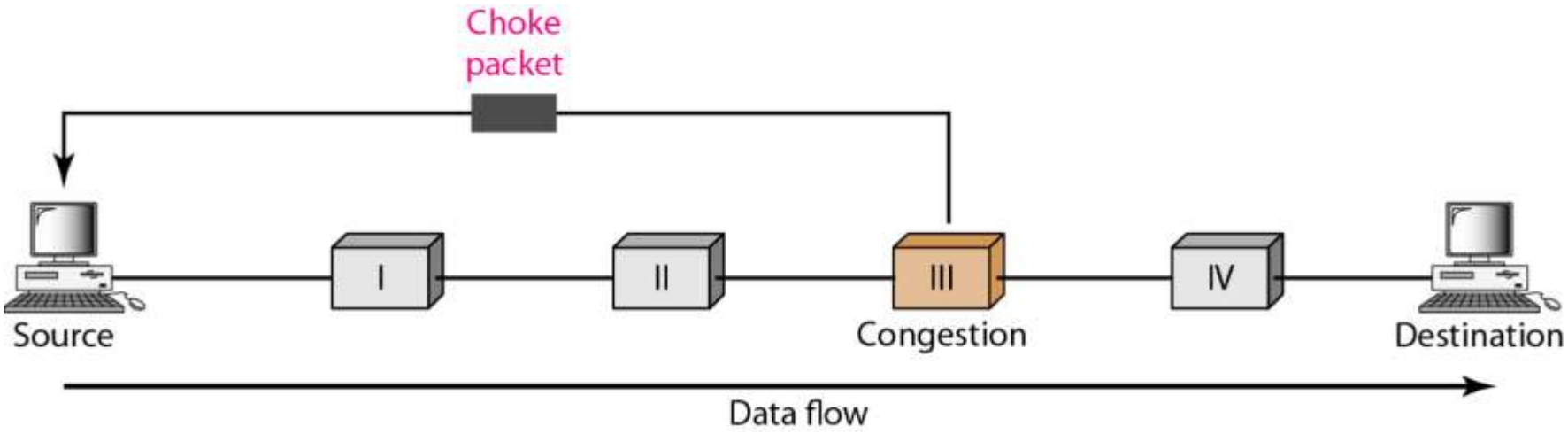
2. Pass information to where action can be taken

Approaches :

1. Send **special packets (choke)** back to its source to inform the problem.
2. Use **bit field** in the packet to indicate that a threshold is exceeded
3. The source sends out **probe packets** at regular intervals to explicitly ask about the congestion.

Closed loop solutions(4)

Sending special packets (choke) back to its source to inform the problem.



- Extra packets increase the load at that moment of time, but reduces the congestion later

Closed loop solutions(5)

3. Adjust system operation to correct the problem

1. Increase resources
   - Increase the bandwidth
   - Increase memory
   - Use a faster processor

2. Decrease load
   - Denying service to some users
   - Degrading service to some users
   - Tell the hosts to reduce their outgoing traffic

Closed loop solutions (6)

Can be divided into two categories.

1. **Explicit approach:** special packets are sent back to the sources to curtail down the congestion
2. **Implicit approach:** the source itself acts pro-actively and tries to deduce the existence of congestion by making local observations

4. Traffic Throttling

In Internet and many other networks, senders adjust their transmissions to send as much traffic as the network can readily deliver. In this setting , the network aims to operate just before the onset of congestion. When congestion is imminent it must tell the senders to throttle back their transmissions and slow down. this can be done in both VC circuit and datagram subnets

- Congestion control in VC Circuits
- Congestion control in Datagram subnets

Congestion Control in Virtual-Circuit Subnets(1)

- Congestion is dynamically controlled in virtual-circuit subnets and includes the following techniques

1. Admission control
2. Select alternative routes
3. Negotiate level of service

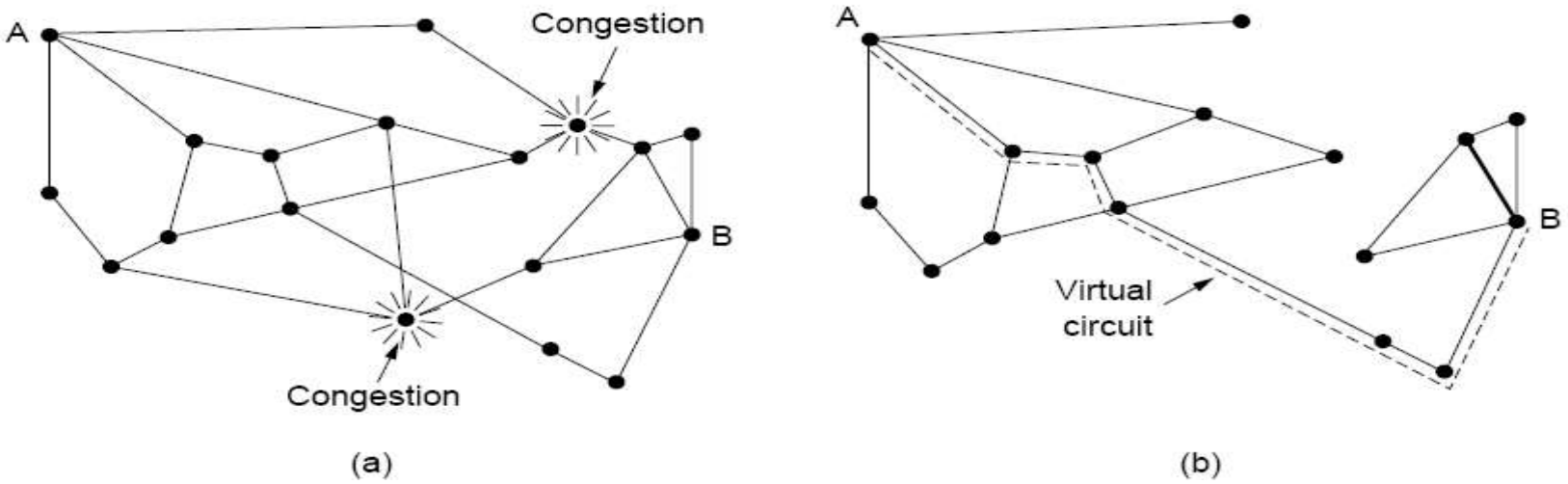
Congestion Control in Virtual-Circuit Subnets(2)

1. Admission control. :

- Once congestion has been signaled, no more virtual circuits are set up until the problem has gone away.
- Thus, attempts to set up new transport layer connections fail.
- Telephone system also uses admission control.

Congestion Control in Virtual-Circuit Subnets(3)

2. **selecting alternative route** : allow new virtual circuits but carefully route all new virtual circuits around problem area.



(a) A congested subnet. (b) A redrawn subnet that eliminates the congestion. A virtual circuit from A to B is also shown.

Congestion Control in Virtual-Circuit Subnets(4

3. Negotiate level of service

- This host normally specifies the volume and shape of the traffic, quality of service required, and other parameters.
- The subnet will agree to this and typically reserve required resources along the path when the circuit is set up
- These resources can include table and buffer space in the routers and bandwidth on the lines.

Congestion Control in Datagram Subnets(1)

- For each output line, router maintains an estimate **u** of its utilization: $u_{new} = au_{old} + (1 - a) f$
  - a a constant.
  - f (0 or1) a sample of the instantaneous line **utilization**, made periodically.

If u_{new} > threshold value , the output line enters a warning state.

- If a packet's output line is in warning state, some action is taken
  1. The warning bit
  2. Choke packets

Congestion Control in Datagram Subnets(2)

1. The Warning Bit

1. Router sets a *special bit* in the packet's header.
2. Destination copies the bit into the next ACK and sends to the source.
3. The sender monitors the number of ACK packets it receives with the warning bit set and adjusts its transmission rate accordingly.
4. As long as the warning bits continue to flow in, the source continues to decrease its transmission rate.

Congestion Control in Datagram Subnets(3)

2. Choke Packets

1. The router in trouble sends a choke packet back to the source host, giving it the destination found in the packet.

1. The original packet is tagged so that it will not generate any more choke packets.

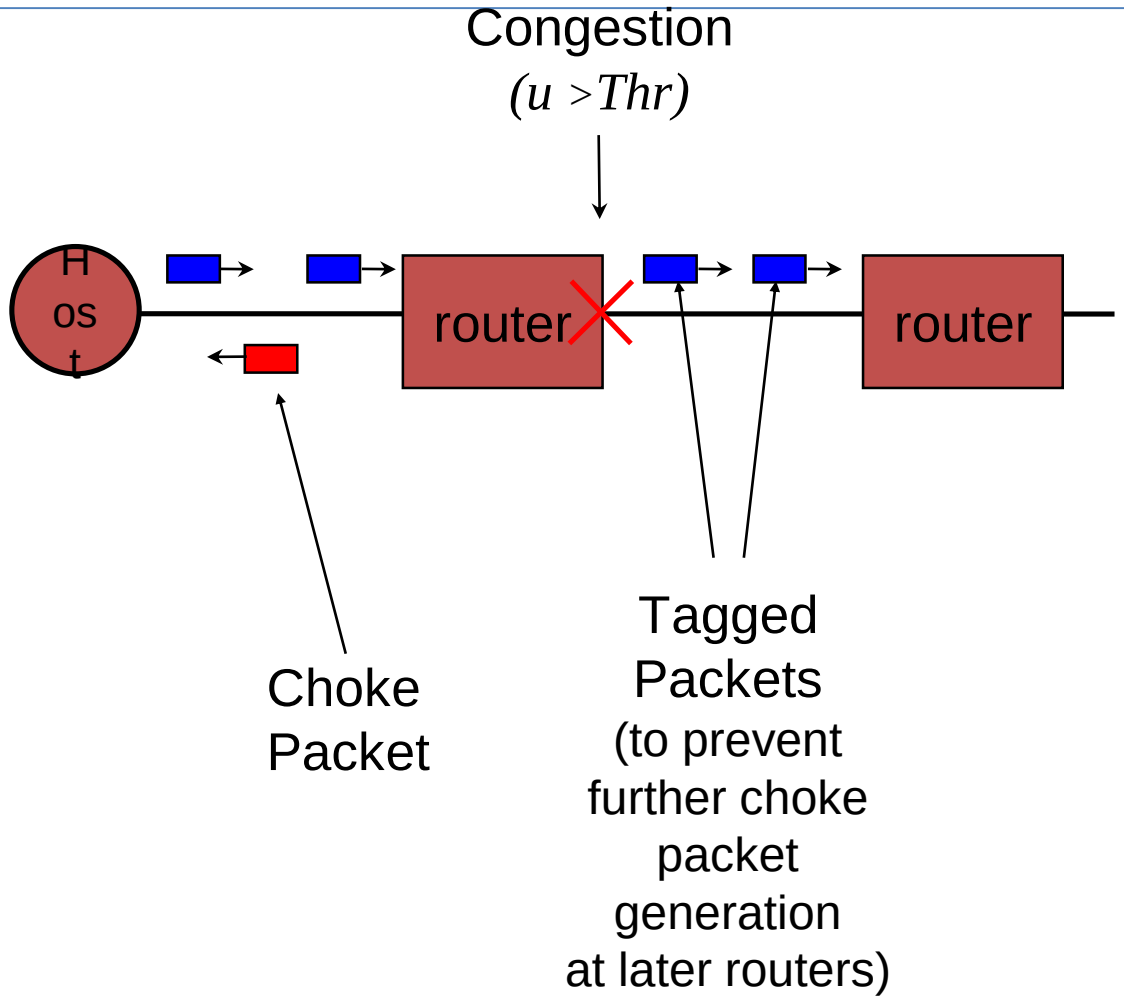
1. When the source gets a choke packet, it cuts rate by half, and ignores further choke packets coming from the same destination for a fixed period

1. After that period has expired, the host listens for more choke packets. If one arrives, the host cut rate by half again. If no choke packet arrives, the host may increase rate

Congestion Control in Datagram Subnets(4)

2. Choke Packets

The original packet is tagged so that it will not generate any more choke packets.



Congestion Control in Datagram Subnets(6)

2. Choke Packets - example

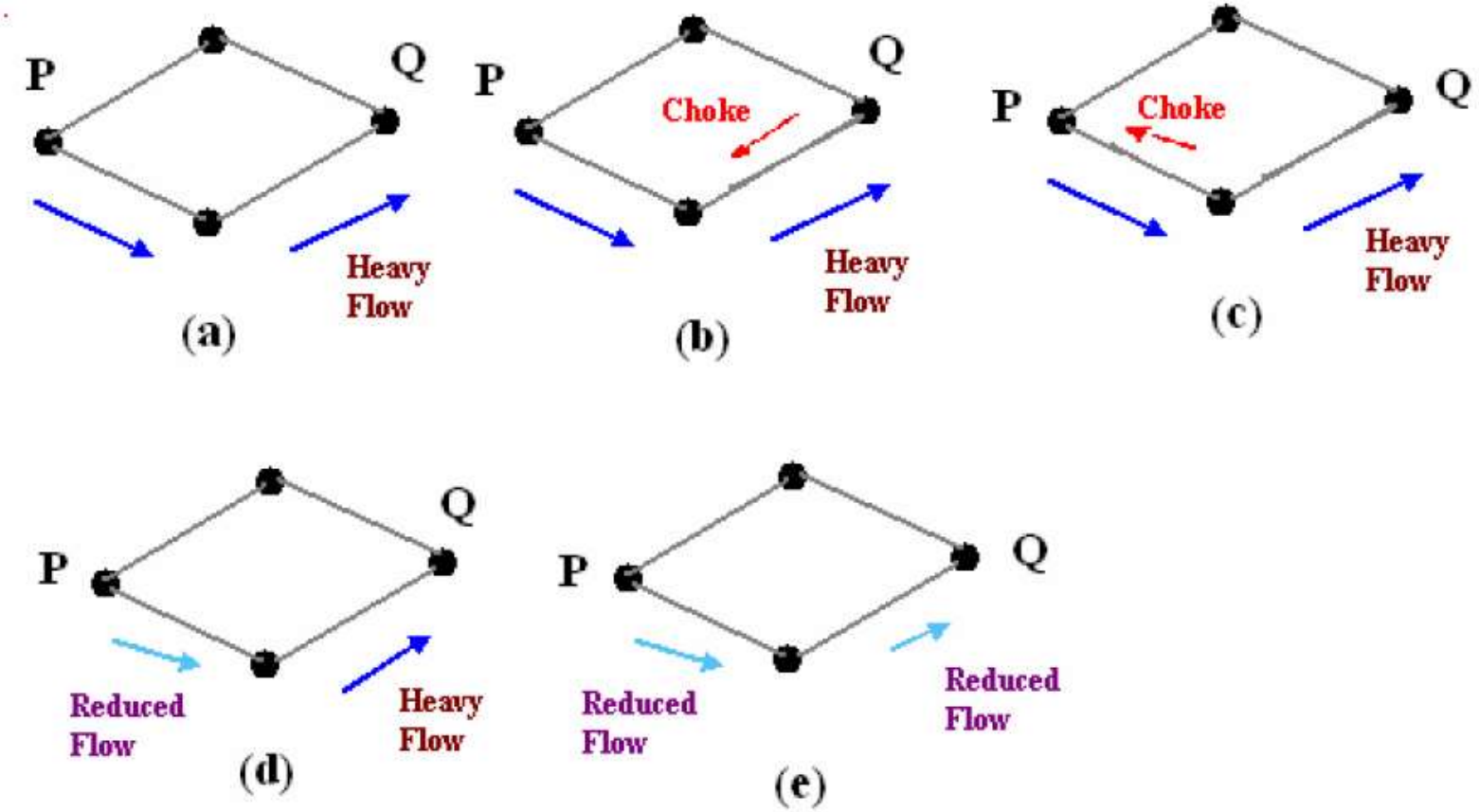
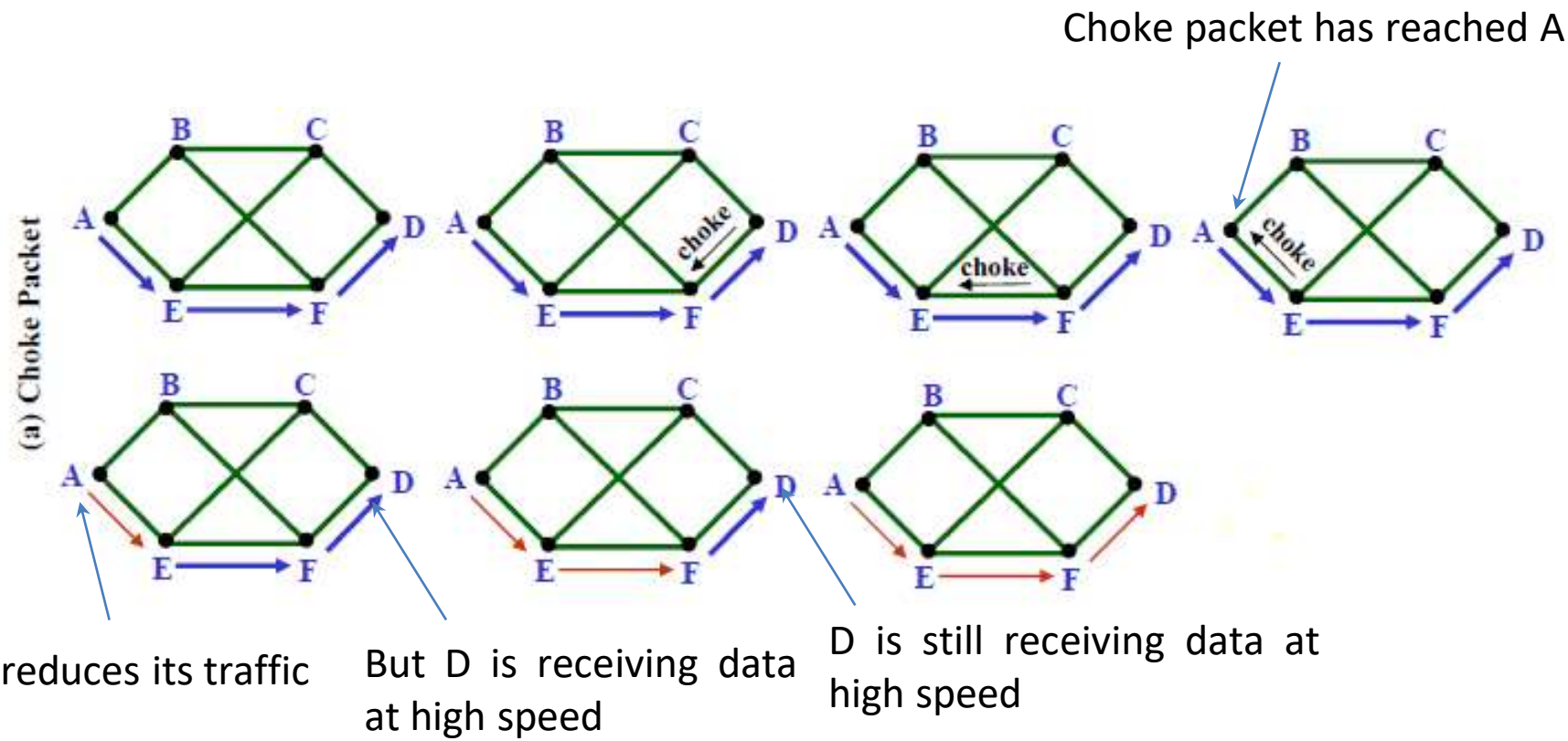


Figure Depicts the functioning of choke packets, (a) Heavy traffic between nodes P and Q, (b) Node Q sends the Choke packet to P, (c) Choke packet reaches P, (d) P reduces the flow and send a reduced flow out, (e) Reduced flow reaches node Q

Congestion Control in Datagram Subnets(7)

Problem with Choke Packet method

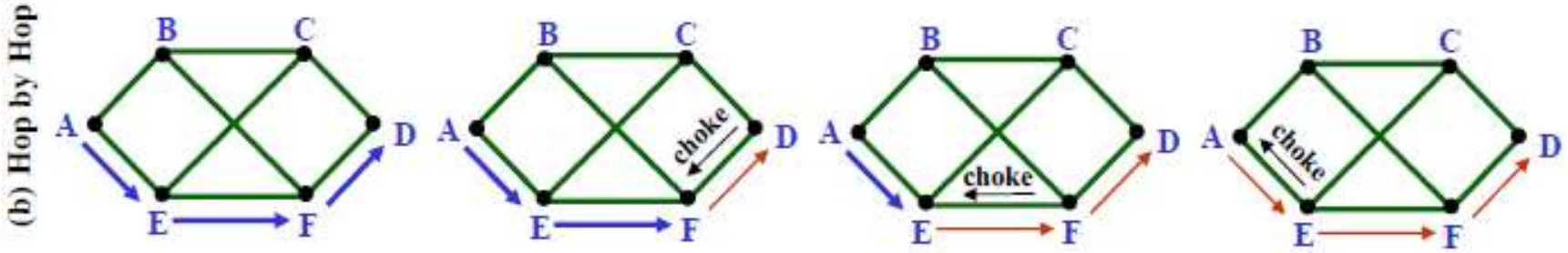
- At high speeds over long distances, this method does not work well because by the time choke packet reach the source, already a lot of packets destined to the same original destination would be out from the source.



Congestion Control in Datagram Subnets(8)

Hop by Hop Choke Packets

-- Mishra and Kanakia, 1992
-- sending a choke packet at high speed over long distance is not effective
-- have the choke packet take effect at every hop it passes through



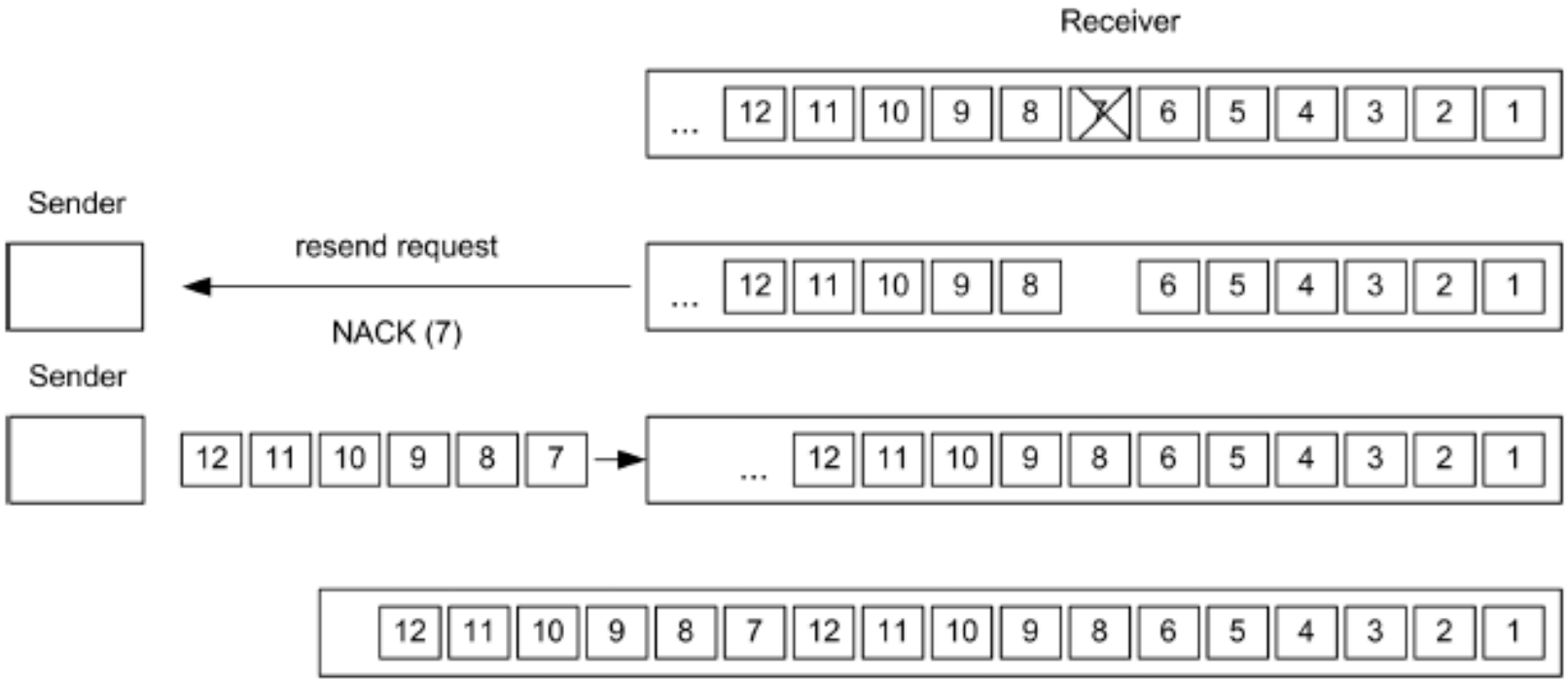
As soon as the choke packet reaches F, F is required to reduce the flow to D. Doing so will require F to devote more buffers to the flow, since the source is still sending away at full blast, but it gives D immediate relief

5. Load Shedding(1)

- A router , when heavily loaded and none of the methods work , can just pick packets AND drop THEM. This is known as **Load Shedding .**
- **Approaches to discard packets**
  - Select packets at random
  - Based on application
    - For File Transfer : keep old, discard new
    - For multimedia : keep new , discard old
  - Intelligence discard policy

5. Load Shedding(2)

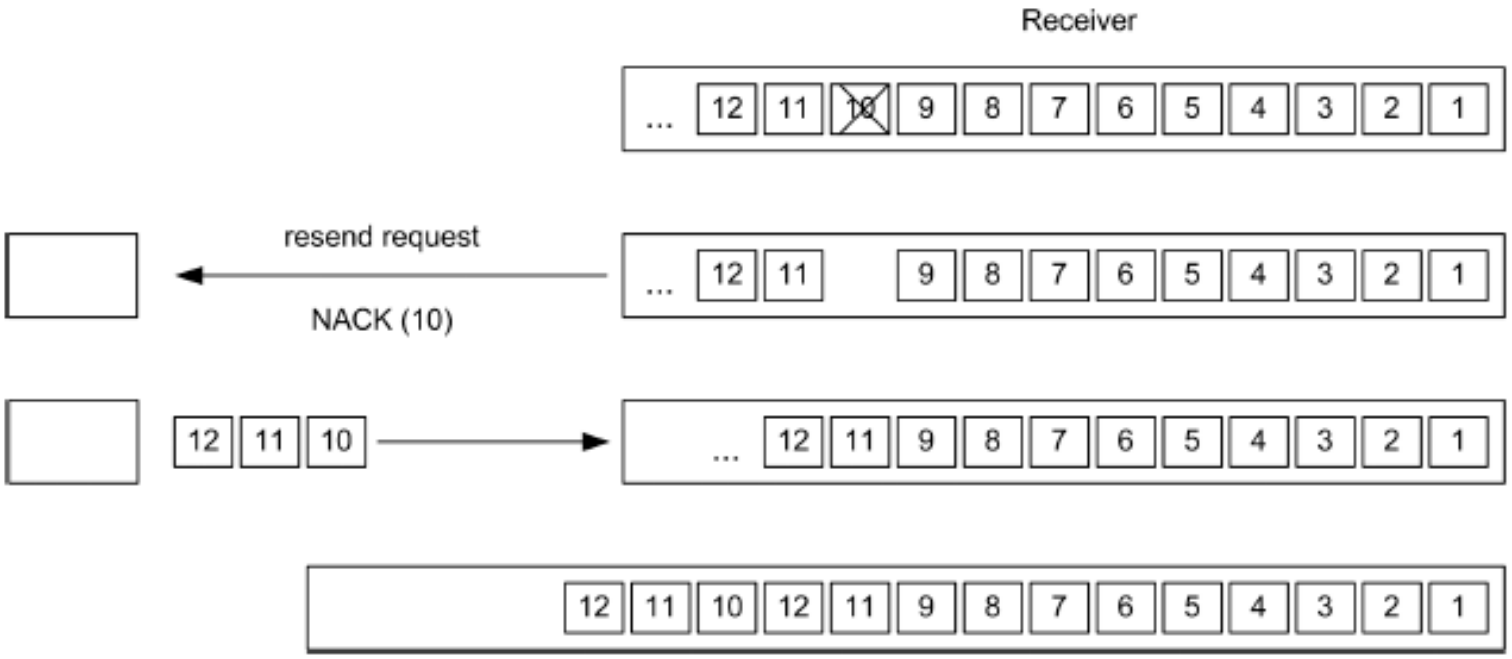
Discarding old packets (as in multimedia)



Discarding packet 7 provokes retransmission of these six packets

5. Load Shedding(3)

Discarding new packets(as in File transfer)



Alternatively, the router can discard packet 10 so that only three packets have to be resend

5. Load Shedding(4)

To implement **intelligent discard policy**,

1. For this , **applications must mark their packets in priority classes**
2. When packets have to be discarded, routers can first drop packets from the lowest class packets, then the next lowest class, and so on

5. Load Shedding(5)

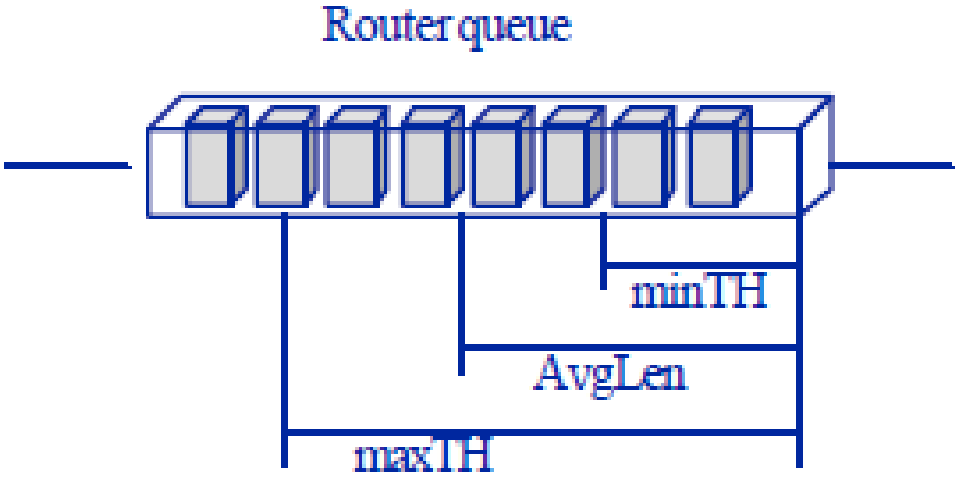
Random Early Detection (RED)

- **A load shedding technique**
  - Idea is to avoid congestion rather than react to it.
  - i.e. Discard packets before all buffer space is exhausted
  - Drop packets before the situation has become hopeless.

5. Load Shedding(6)

Random Early Detection (RED)

- Routers maintain a running average of their queue lengths . When the average queue length on some line exceeds a threshold, the line is said to be congested and action is taken.



5. Load Shedding(7)

Random Early Detection (RED)

The RED algorithm

When a new packet arrives , calculate of the average queue length ,**avg_qlen,** compare it with two thresholds **THmin** and **THmax**.

If avg_qlen < THmin,

queue packet;

If THmin ≤ avg_qlen < THmax,

calculate probability P, with probability P discard packet, with probability (1-P) queue packet;

If avg_qlen ≥ THmax,

pick packet at random and discard .

5. Load Shedding(8)

Random Early Detection (RED)

How should the router tell the source about the problem?

1. Send it a choke packet

puts even more load on the already congested network

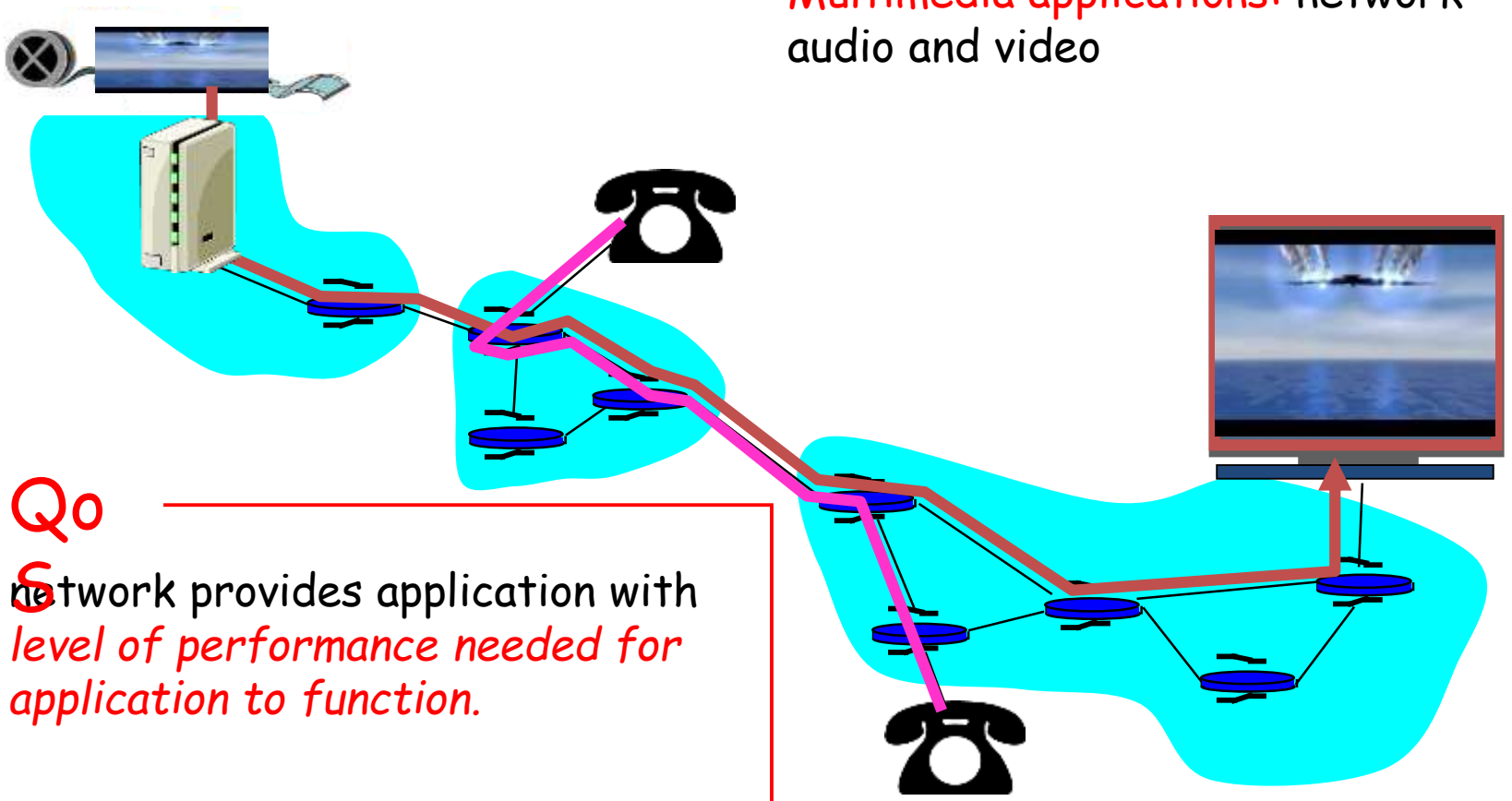
1. Discard the selected packet and not report it

When a source notices the lack of ACK, it reduces transmission rate.

Quality of Service

Why Quality of Service?

Multimedia applications: network audio and video



QoS

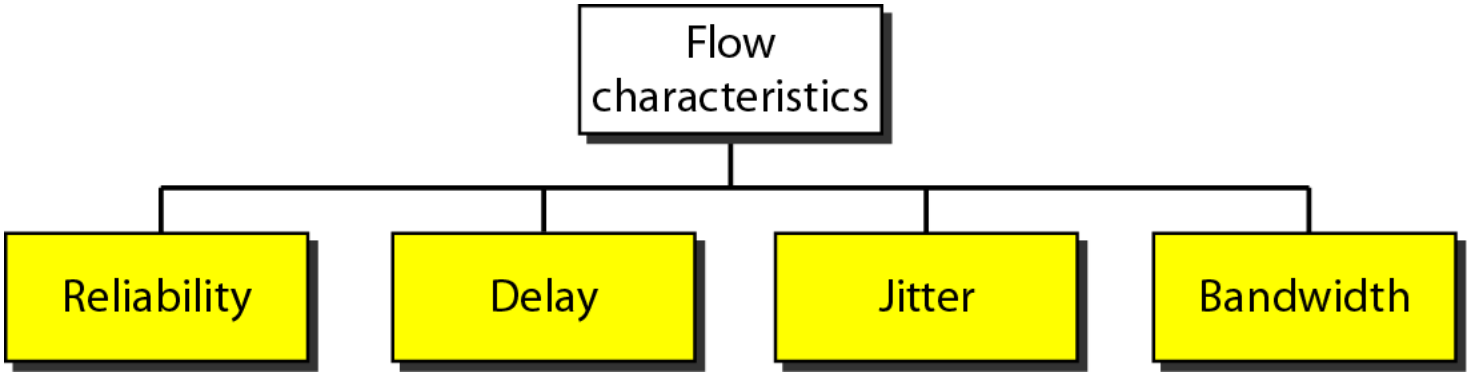
network provides application with *level of performance needed for application to function.*

Why Quality of Service?

- **Growth of multimedia networking**
- Congestion control techniques
  - Can reduce congestion and improve network performance
  - **But may not achieve QOS for multimedia traffic.**
  - **so we need other alg. in addition to Congestion control Tech. to achieve QOS.**

What is a Flow?

- A stream of packets from a source to a destination is called a **flow.**
- A Flow needs 4 characteristics:



Together these determine the QoS (Quality of Service) the flow requires.

Quality-of-Service Requirements

Different applications need different QOS

Application	Reliability	Delay	Jitter	Bandwidth
E-mail	High	Low	Low	Low
File transfer	High	Low	Low	Medium
Web access	High	Medium	Low	Medium
Remote login	High	Medium	Medium	Low
Audio on demand	Low	Low	High	Medium
Video on demand	Low	Low	High	High
Telephony	Low	High	High	Low
Videoconferencing	Low	High	High	High

Techniques for Achieving Good Quality of Service

1. **Traffic Shaping**
   - The Leaky Bucket Algorithm
   - The Token Bucket Algorithm
2. **Admission Control**
3. **Packet Scheduling**

1. Traffic Shaping

- If the source outputs the packets at non - uniform rate, congestion may occur in the network.

- Traffic shaping is a mechanism to control the amount and the rate of the traffic sent to the network.

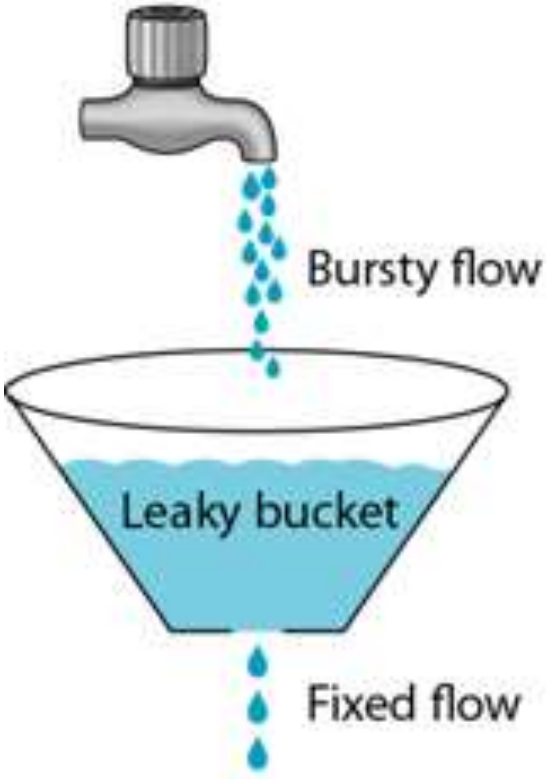
- **Traffic shaping, smooths out the traffic on the server side, rather than on the client side.**

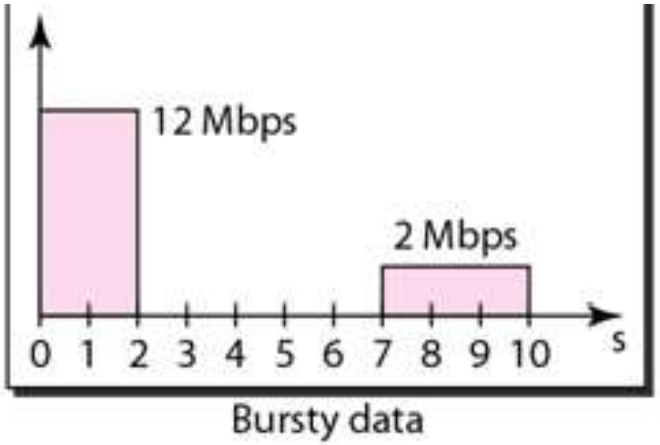
1. Traffic Shaping

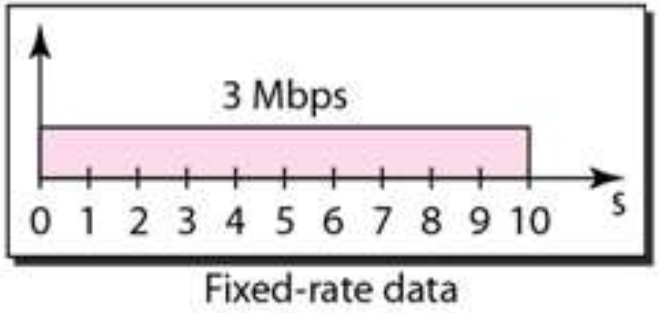
- **Traffic shaping Algorithms**:
    1. leaky bucket Algorithm
        - first proposed by **Jonathan S. Turner ,1986**
        - Used in ATM networks
    2. Token bucket Algorithm

Leaky Bucket Algorithm

The leaky bucket enforces a constant output rate (average rate) regardless of the burstiness of the input.



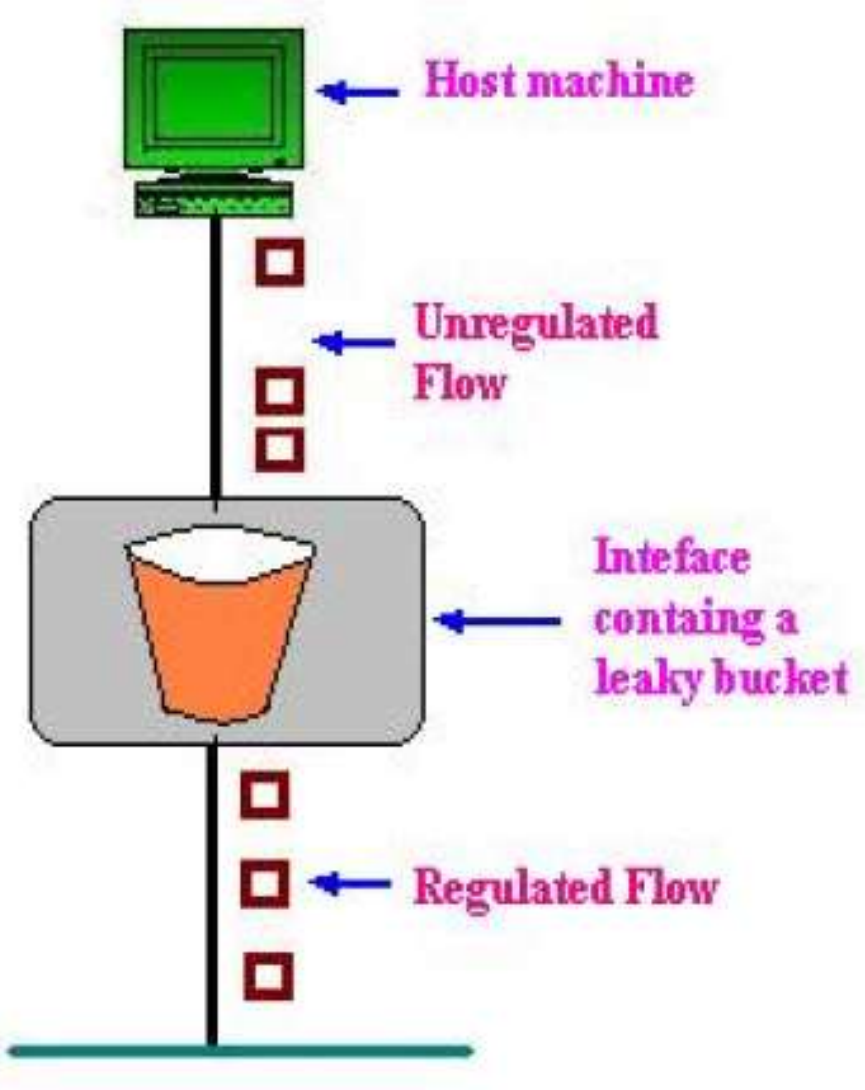




It drops the packets if the bucket is full

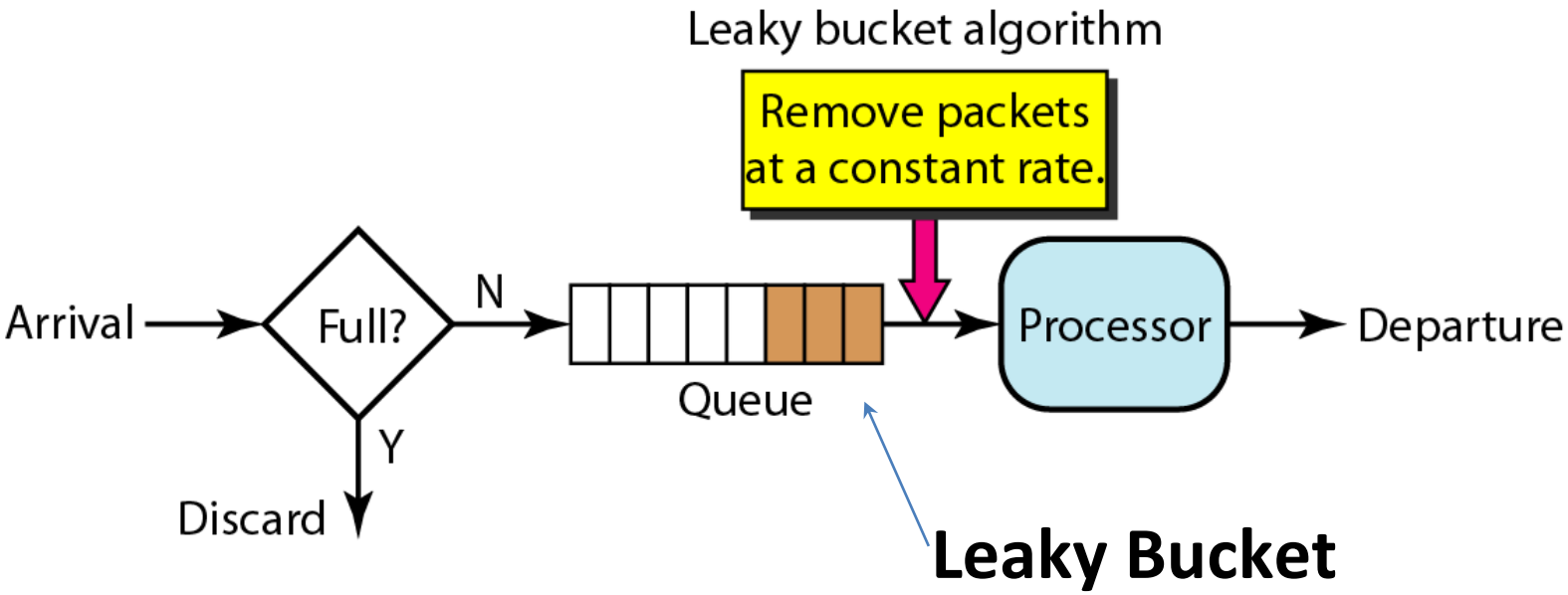
Leaky Bucket Algorithm

- Either built into the network hardware interface or implemented by the operating system.
- Conceptully the bucket is a finite internal queue



Leaky Bucket Implementation

- **Algorithm for fixed -length packets**
  1. The leaky bucket consists of a finite queue.
  2. When a packet arrives, if there is room on the queue it is appended to the queue; otherwise, it is discarded.
  3. At every clock tick, one packet is transmitted



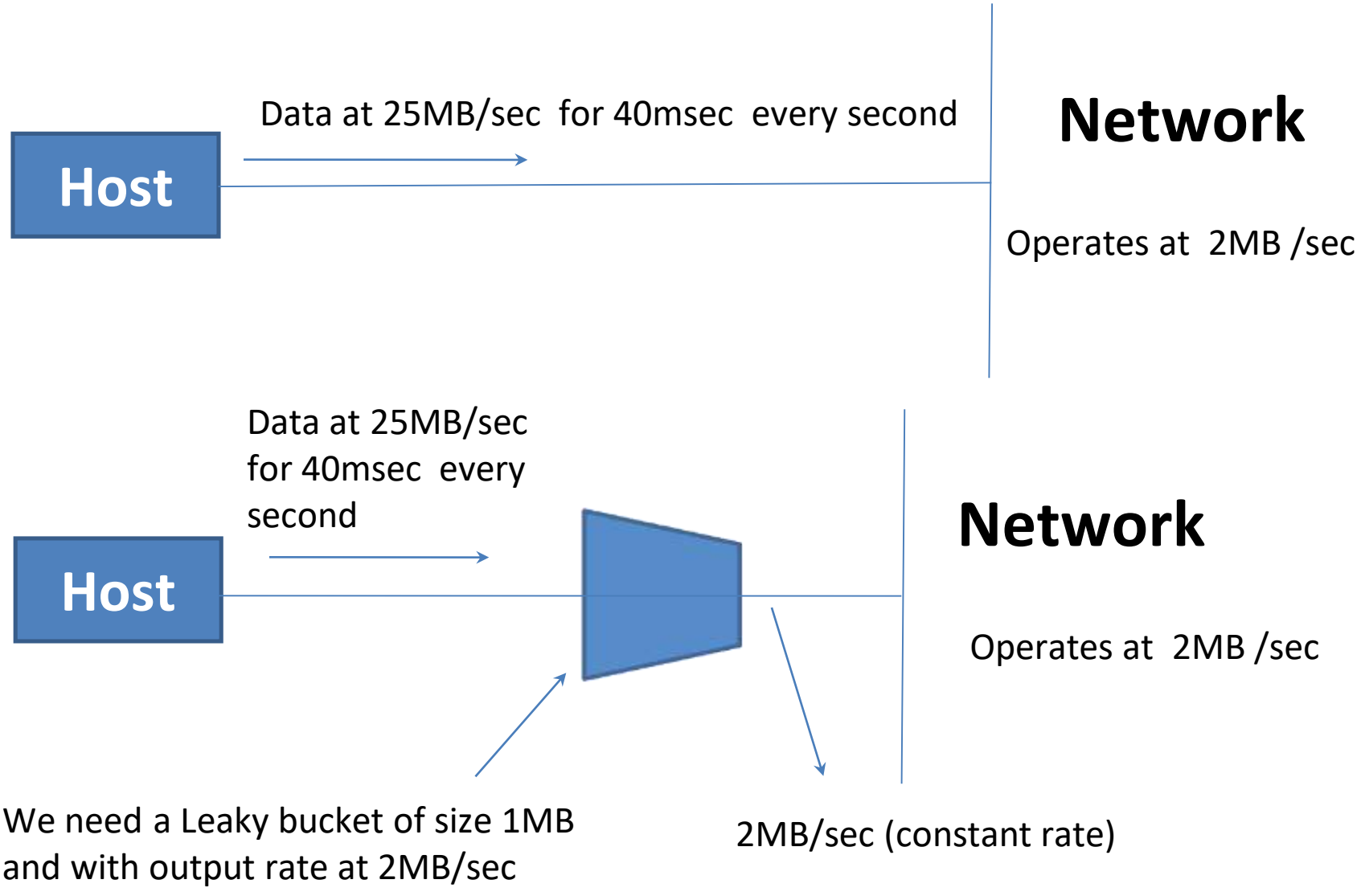
Leaky Bucket Implementation

- **Algorithm for variable -length packets**

For variable length packets though, allow a fixed number of bytes per tick

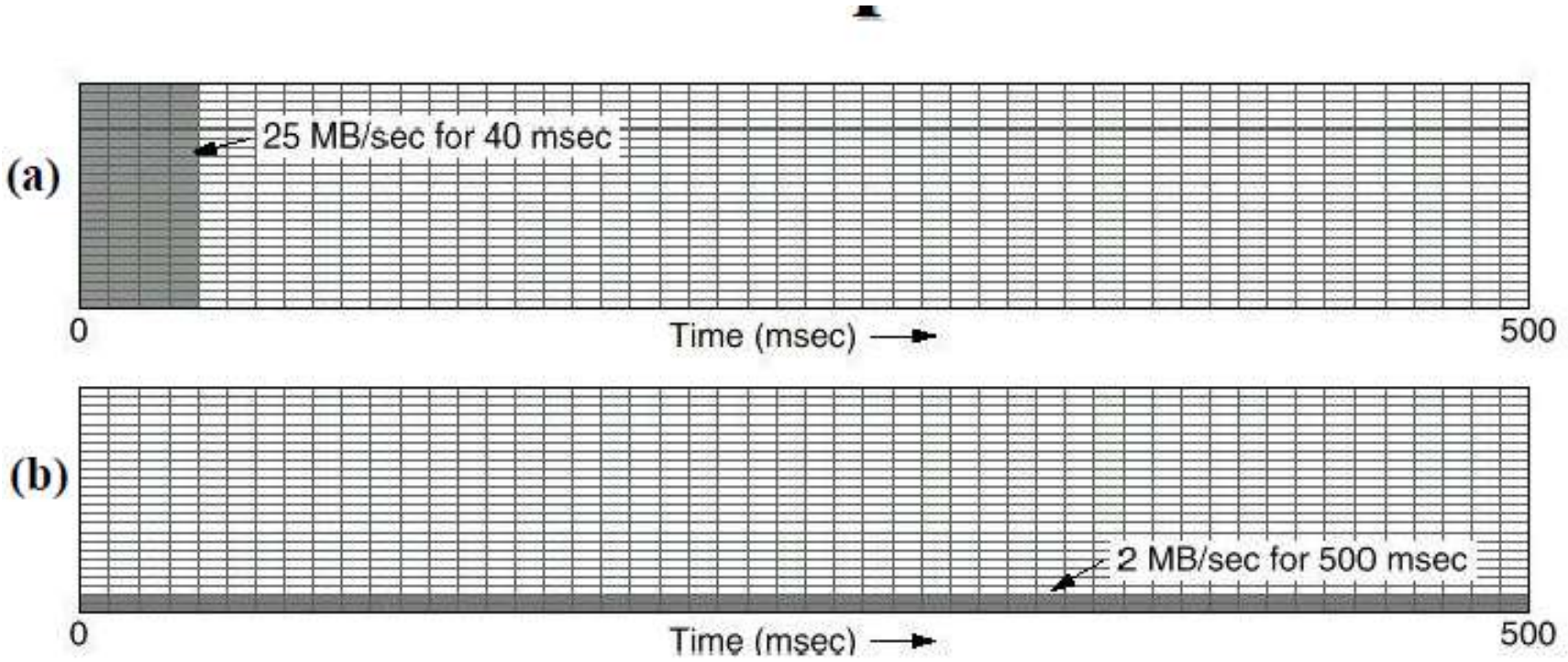
1. Initialize a counter to n at the tick of the clock
2. If n is greater than the size of the packet, send packet and decrement the counter by the packet size. Repeat this step until n is smaller than the packet size
3. Reset the counter and go to step 1

Leaky Bucket – Example



Leaky Bucket – Example

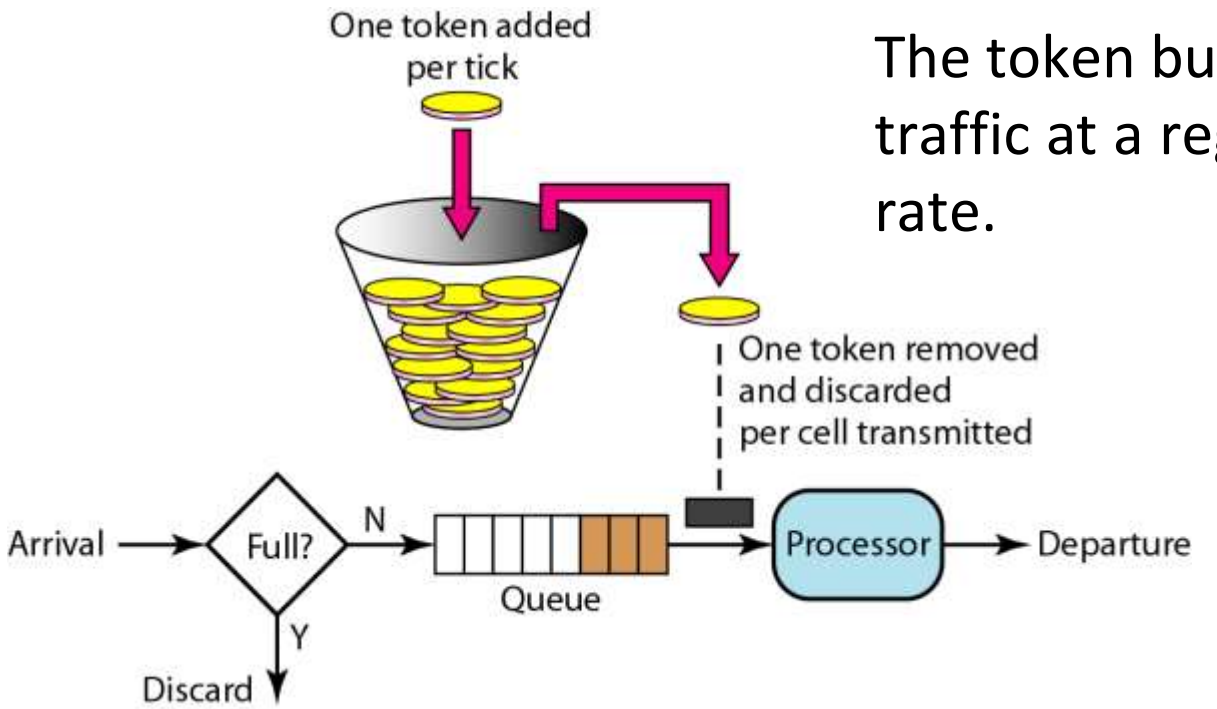
the output drains out at a uniform rate of 2 MB/sec for 500 msec.



(a) Input to a leaky bucket from host
(b) Output from a leaky bucket.

The Token Bucket Algorithm

- For many applications, it is better to allow the output to speed up somewhat when large bursts arrive.

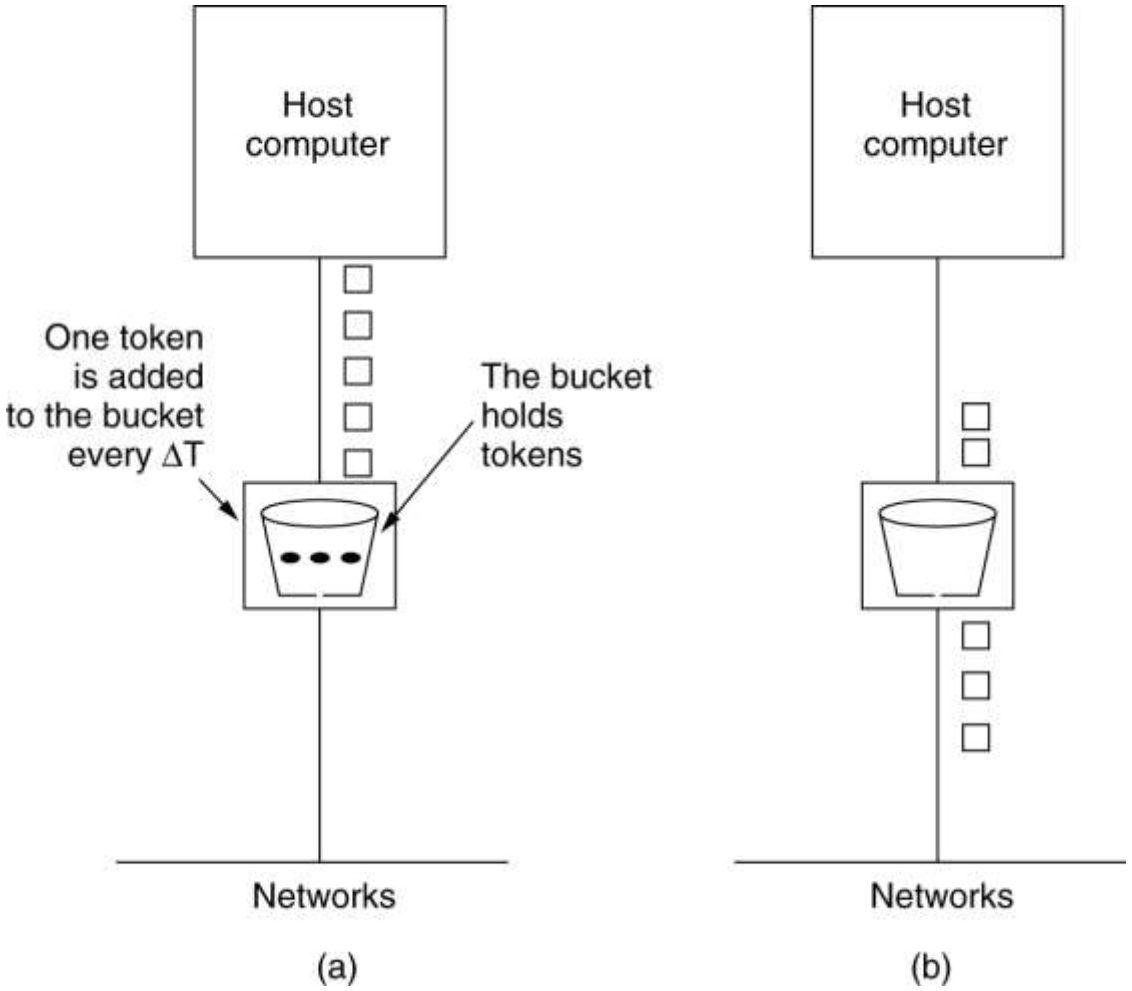


The token bucket allows bursty traffic at a regulated maximum rate.

For a packet to be transmitted, it must capture and destroy one token.

The Token Bucket Algorithm

The token bucket algorithm. (a) Before. (b) After



The Token Bucket Algorithm

The maximum output rate (m) is given by :

$$m = c/s + \rho$$

c – the token bucket capacity

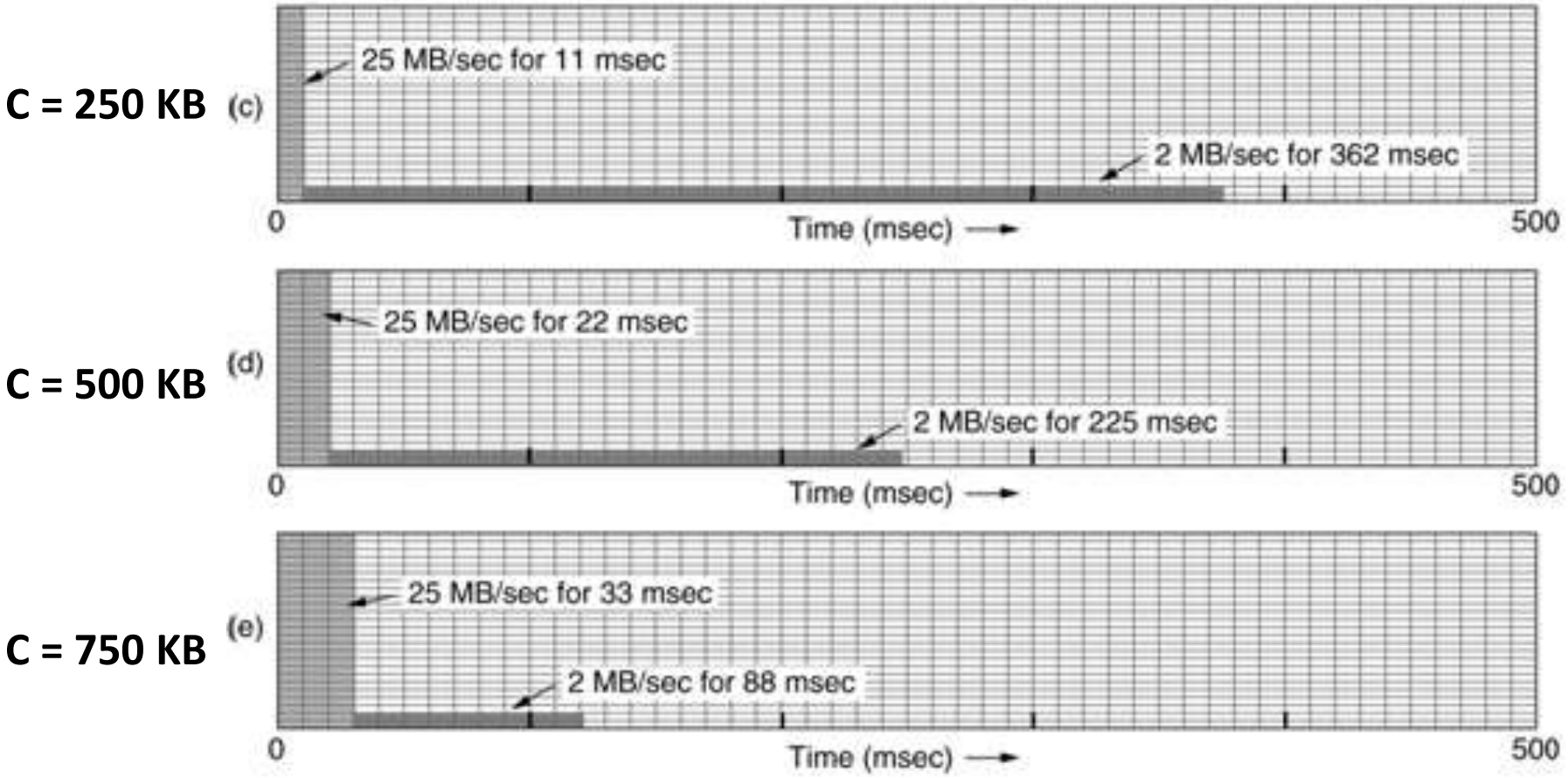
s – the burst length in seconds

ρ – the token generating rate

The Token Bucket – Example

M = 25 MB/sec, and ρ =2 MB/sec

Output from token bucket



The Token Bucket Algorithm

The Token Bucket Algorithm:

1. bucket := ∅; token = 0;
   $m := c/s+\rho$, where m is the max output rate, c is the token bucket capacity, s is the burst length in seconds, ρ is the token generating rate.
2. repeat
   if (token < c) accept packets
   if (token > 0)
     p := dequeue (bucket, m)
     output p
   if (t == Δt)
     token++
   if (requested) adjust m
   end repeat

Token Bucket vs. leaky Bucket

Token

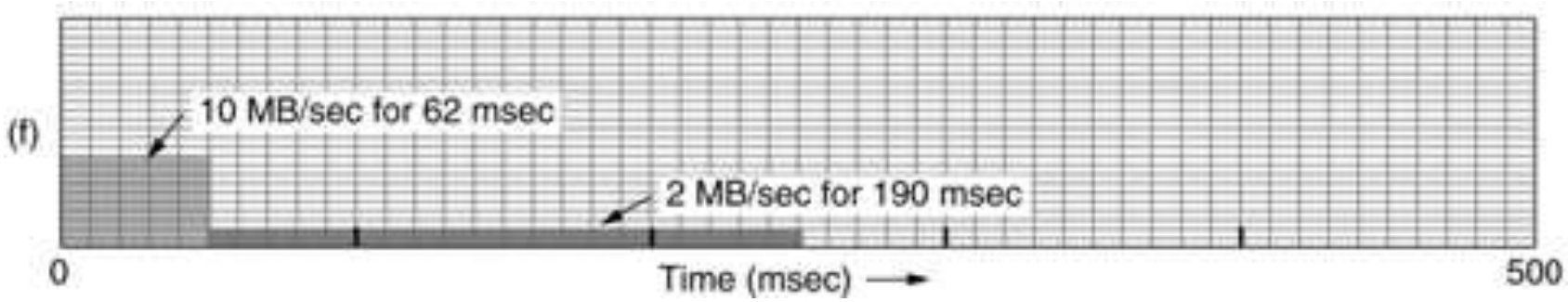
- allows the output rate to vary
- bucket holds tokens
- Idle hosts can capture and save up tokens
- never discards packets when bucket is full

Leaky

- enforces a constant output rate
- bucket holds data
- Does not allow idle hosts to save up permissions
- discards packets when bucket is full

Combined Token - leaky Bucket

- A potential problem with the token bucket algorithm is that it allows large bursts again
- To overcome this , insert a leaky bucket after the token bucket
- The rate of the leaky bucket should be higher than the token bucket's ρ but lower than the maximum rate of the network



Output from a 500KB token bucket feeding a 10-MB/sec leaky bucket.

Exercise(TBA,LBA)

1. A source generates data in terms of bursts: 3 MB bursts lasting 2 msec once every 100 msec. The network offers a bandwidth of 60 MB/sec. The leaky bucket has a capacity of 4 MB.
   a. How does the output look like?
   b. What should be the minimum capacity of the leaky bucket to avoid loss?

Exercise(TBA,LBA)

2. An ATM network uses a token bucket scheme for traffic shaping. A new token is put into the bucket every 5 µsec. Each token is good for one cell, which contains 48 bytes of data. What is the maximum sustainable data rate?

2. A computer on a 6-Mbps network is regulated by a token bucket. The token bucket is filled at a rate of 1 Mbps. It is initially filled to capacity with 8 megabits. How long can the computer transmit at the full 6 Mbps?

2. Admission Control

- When a flow is offered to a router, it has to decide whether to admit or reject the flow.
- The decision is based on
  1. its capacity
  2. how many commitments it has already made for other flows.

2. Admission Control(2)

Flow Specification

- A set of flow parameters is called a **flow specification**

1. sender produces a flow specification
2. As the specification propagates along the route, each router examines it and modifies the parameters.
3. When it gets to the other end, the parameters can be established.

2. Admission Control(3)

An example Flow Specification

Parameter	Unit
Token bucket rate	Bytes/sec
Token bucket size	Bytes
Peak data rate	Bytes/sec
Minimum packet size	Bytes
Maximum packet size	Bytes

Peak rate: Defines the maximum rate at which packets can be sent in short interval time.

3. Admission Control(4)

- **Mapping a flow specification into a set of specific resource reservations is implementation specific**

Example :

router capacity =100,000 packets/sec

flow rate = 1 MB/sec

minimum and maximum packet sizes = 512 bytes

the router can calculate that it might get 2048 packets/sec from that flow.

So it must reserve 2% of its CPU capacity for that flow

2.Admission Control(5)

- **Tight flow specification is more useful to the routers**

Example:

Assume router capacity =100,000 packets/sec

inaccurate flow specification :

1. token bucket rate = 5 MB/sec
2. packet can vary from 50 - 1500 bytes

the router can calculate that the packet rate will vary from about 3500 packets/sec to 105,000 packets/sec

So the router may reject the flow.

Exercise

- The CPU in a router can process 2 million packets/sec. The load offered to it is 1.5 million packets/sec. If a route from source to destination contains 10 routers, how much time is spent being queued and serviced by the CPUs?

3. Packet Scheduling

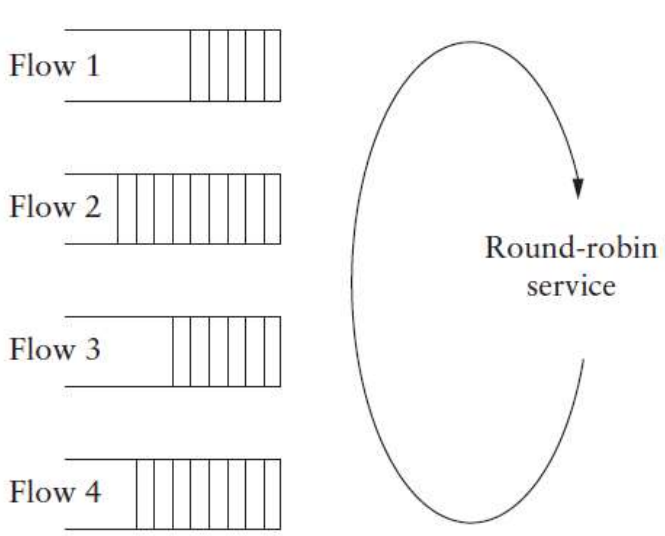
- If a router is handling multiple flows, one flow might use too much of its capacity and starve other flows .
- To avoid such attempts, various packet scheduling algorithms have been devised.

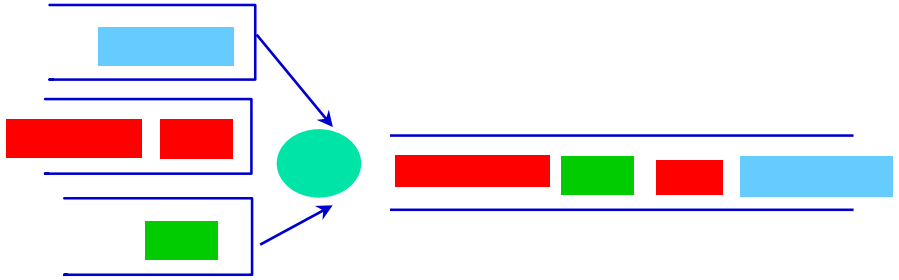
1. **Fair queuing algorithm** (Nagle, 1987).
2. **Weighted fair queueing**

3. Packet Scheduling

Fair queuing

- Routers have multiple queues for each output line, one for each source.
- Take first packet in each queue on a round-robin basis





packet-by-packet round robin

3. Packet Scheduling

Fair queuing Problem

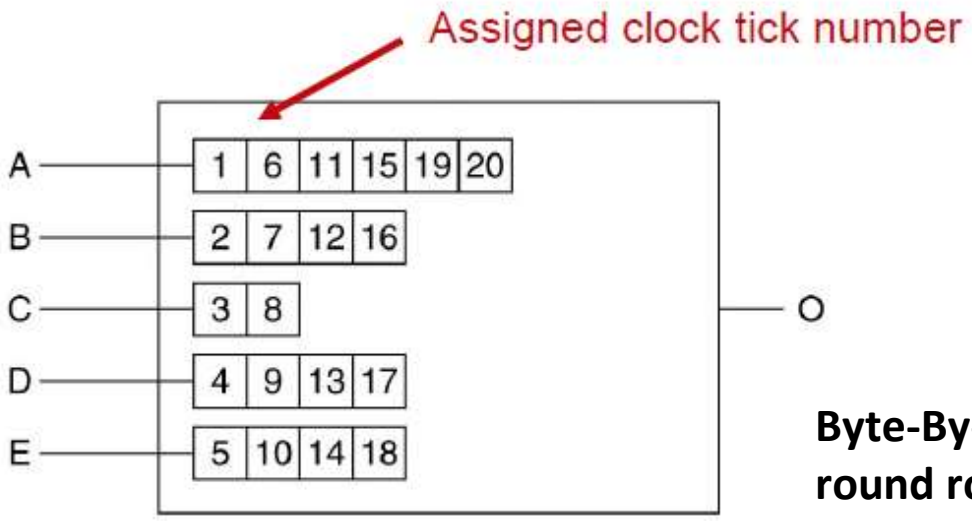
- It gives more bandwidth to hosts that use large packets than to hosts that use small packets

- **Demers et al. (1990) suggested an improvement .**
  - simulate a byte-by-byte round robin, instead of a packet-by-packet round robin .

3. Packet Scheduling

Fair queuing : An improvement.

- Scan queues repeatedly until tick is found at which packet is done
- Sort them in order of their finishing and send in that order



(a)

Byte-By-byte round robin

Packet	Finishing time
C	8
B	16
D	17
E	18
A	20

(b)

(a) A router with five packets queued for line O. (b) Finishing times for the five packets

3. Packet Scheduling

Fair queuing : An improvement.

- Problem : it gives all hosts the same priority
- In many situations, it is desirable to give video servers more bandwidth than regular file servers
- This modified algorithm is called **weighted fair queueing**

3. Packet Scheduling

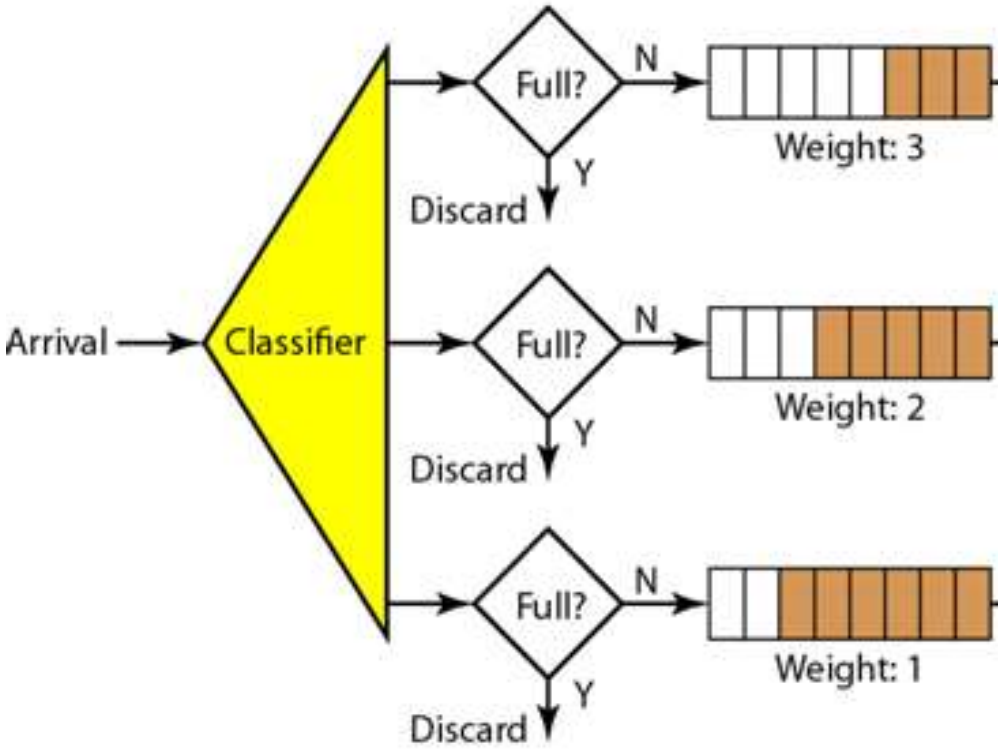
Weighted Fair Queuing.

- Packets are assigned to different classes and admitted to different queues.
- The queues are weighted based on the priority of the queues.
  - *higher priority means a higher weight*

3. Packet Scheduling

Weighted Fair Queuing.

- The system processes packets in each queue in a round-robin fashion with number of packets selected from each queue based on the corresponding weight.



if the weights are 3, 2, and 1, three packets are processed from the first queue, two from the second queue, and one from the third queue